

Universidad Politécnica de Querétaro

Programa Educativo:
Ingeniería en Sistemas Computacionales

PREDIA

Plataforma Clínica Inteligente para la Detección Temprana de
Diabetes

REPORTE DE PROYECTO INTEGRADOR

Integración móvil para autogestión del paciente y atención clínica en
movilidad

Informe técnico:

Configuración de variables de entorno del proyecto PREDIA

Presentan:

Arianna Valentina Giannoccaro Quiñonez
Álvarez Sánchez Guillermo
Muñoz Prado Christopher Yanhyu
Villafuerte Armenta Gabriel Iván

Asignatura: Programación Móvil

Junio de 2026

Índice

1	Objetivo	2
2	Apuntes	2
2.1	¿Qué son las variables de entorno?	2
2.2	Archivos de entorno identificados	2
2.3	Variables principales	3
3	Desarrollo de la configuración	3
3.1	Paso 1: localizar los archivos de entorno	3
3.2	Paso 2: crear el archivo local a partir de la plantilla	4
3.3	Paso 3: configurar backend y base de datos	5
3.4	Paso 4: configurar variables de seguridad	6
3.5	Paso 5: configurar la app móvil Expo	7
3.6	Paso 6: validar la configuración	8
3.7	Paso 7: aplicar buenas prácticas	9
4	Resultado final	10
5	Conclusión	10

1 Objetivo

Documentar de forma clara y verificable cómo se configuraron las variables de entorno del proyecto PREDIA, explicando su propósito, los archivos utilizados, los pasos de configuración, las medidas de seguridad aplicadas y la validación técnica realizada para comprobar que el backend web, la aplicación móvil y la base de datos pueden comunicarse correctamente.

El informe se enfoca en la configuración necesaria para trabajar con PREDIA en un entorno local de desarrollo. Por seguridad, las credenciales reales, contraseñas, tokens y secretos se muestran censurados en las capturas y ejemplos.

Nota de seguridad

Las capturas incluidas documentan los pasos realizados y muestran únicamente valores seguros o censurados. No se exponen contraseñas reales, cadenas completas con credenciales ni el valor original de `JWT_SECRET`.

2 Apuntes

2.1 ¿Qué son las variables de entorno?

Las variables de entorno son valores externos al código fuente que permiten configurar una aplicación sin modificar sus archivos internos. En PREDIA se usan para definir la conexión a MySQL, los secretos de autenticación JWT, URLs de API, parámetros de ejecución y direcciones que necesita Expo para consumir el backend desde la app móvil.

Este enfoque permite separar el código de los datos sensibles. Así, el repositorio puede compartirse con plantillas como `.env.example`, mientras que cada integrante conserva sus credenciales reales en archivos locales como `.env.local` o `apps/mobile/.env`.

2.2 Archivos de entorno identificados

Durante la revisión del proyecto se localizaron los siguientes archivos:

Archivo	Uso dentro del proyecto
<code>.env.example</code>	Plantilla general compartible. Contiene nombres de variables y valores de ejemplo sin credenciales reales.
<code>.env.local</code>	Archivo local para desarrollo. Contiene variables reales del equipo, por lo que no debe compartirse públicamente.
<code>.env.production</code>	Archivo destinado a despliegue productivo, con URLs y secretos propios de producción.
<code>apps/web/.env</code>	Variables específicas del backend Next.js y Prisma.
<code>apps/mobile/.env</code>	Variables que Expo expone al bundle móvil, principalmente <code>EXPO_PUBLIC_API_URL</code> .
<code>apps/mobile/.env.example</code>	Plantilla para configurar la URL de API según emulador, navegador o dispositivo físico.

2.3 Variables principales

Variable	Módulo	Propósito
DATABASE_URL	Backend / Prisma	Cadena de conexión MySQL usada por Prisma y servicios de base de datos.
DATABASE_HOST	Backend	Host alternativo cuando se configura la base por variables separadas.
DATABASE_PORT	Backend	Puerto de MySQL, normalmente 3306.
DATABASE_NAME	Backend	Nombre de la base de datos, en este caso predia .
DATABASE_USER	Backend	Usuario con permisos sobre la base.
DATABASE_PASSWORD	Backend	Contraseña del usuario de base de datos. Debe permanecer privada.
JWT_SECRET	Seguridad	Llave secreta para firmar y verificar tokens de sesión.
JWT_EXPIRES_IN	Seguridad	Tiempo de expiración del token de acceso.
NEXT_PUBLIC_API_URL	Web	URL pública usada por el frontend web para consumir API.
ML_API_URL	Integración IA	Dirección del servicio de Machine Learning si se ejecuta separado.
EXPO_PUBLIC_API_URL	Móvil	URL que la app Expo usa para comunicarse con el backend.
NODE_ENV	Web	Define si el entorno es desarrollo o producción.

3 Desarrollo de la configuración

3.1 Paso 1: localizar los archivos de entorno

Primero se identificaron los archivos `.env` existentes en la raíz del repositorio y dentro de las aplicaciones `apps/web` y `apps/mobile`. Esto permitió distinguir qué variables corresponden al backend, cuáles son de la app móvil y cuáles son plantillas.

Paso 1. Localización de archivos de entorno

Se identificaron las plantillas y archivos locales usados por PREDIA.

```
terminal · /home/wowo/Descargas/predia

$ find . -maxdepth 4 -name ".env*" -type f | sort
./env
./env.example
./env.local
./env.production
./apps/mobile/.env
./apps/mobile/.env.example
./apps/web/.env

# Se separan variables del backend web, móviles y producción.
# Los secretos reales no se suben ni se documentan en texto plano.
```

Figura 1: Localización de archivos de entorno dentro del repositorio PREDIA.

3.2 Paso 2: crear el archivo local a partir de la plantilla

La configuración local se preparó tomando como referencia `.env.example`. La plantilla documenta las variables esperadas y evita que el equipo tenga que adivinar nombres o formatos.

Listing 1: Comando base para preparar el archivo local.

```
cp .env.example .env.local
```

Después se editó `.env.local` para establecer el puerto de desarrollo y la URL local de API.

Paso 2. Creación del archivo local

Se tomó la plantilla `.env.example` como base para el entorno de desarrollo.

```
terminal · preparación

$ cp .env.example .env.local
$ nano .env.local

# Archivo local usado durante desarrollo:
NODE_ENV="development"
PORT=3002
NEXT_PUBLIC_API_URL="http://127.0.0.1:3002/api"

# Valores sensibles se mantienen fuera del repositorio.
# .env.local queda en el equipo del desarrollador.
```

Figura 2: Creación del archivo `.env.local` desde la plantilla del proyecto.

3.3 Paso 3: configurar backend y base de datos

PREDIA utiliza MySQL como base de datos y Prisma como capa de acceso a datos. Por ello, la variable más importante para persistencia es `DATABASE_URL`. Además, el proyecto incluye variables separadas como `DATABASE_HOST`, `DATABASE_PORT`, `DATABASE_USER`, `DATABASE_PASSWORD` y `DATABASE_NAME`, que sirven como respaldo cuando se desea configurar la conexión por partes.

Listing 2: Ejemplo censurado de variables de base de datos.

```
DATABASE_HOST="127.0.0.1"
DATABASE_PORT="3306"
DATABASE_NAME="predia"
DATABASE_USER="predia_user"
DATABASE_PASSWORD="*****"
DATABASE_URL="mysql://predia_user:*****@127.0.0.1:3306/predia"
```

Paso 3. Configuración del backend y base de datos

Next.js, Prisma y las rutas API usan DATABASE_URL o variables individuales.

```
editor · .env.local

# DATABASE (MySQL)
DATABASE_HOST="127.0.0.1"
DATABASE_PORT="3306"
DATABASE_NAME="predia"
DATABASE_USER="predia_user"
DATABASE_PASSWORD="*****"
DATABASE_URL="mysql://predia_user:*****@127.0.0.1:3306/predia"

# Consumo interno del frontend web
NEXT_PUBLIC_API_URL="http://127.0.0.1:3002/api"
ML_API_URL="http://127.0.0.1:8000"
```

Figura 3: Configuración censurada de variables de backend y base de datos.

3.4 Paso 4: configurar variables de seguridad

Para la autenticación de usuarios, PREDIA utiliza tokens JWT. La variable `JWT_SECRET` debe ser larga, aleatoria y privada. Se generó con un comando criptográficamente seguro y se almacenó únicamente en el archivo local.

Listing 3: Generación recomendada de un secreto JWT.

```
openssl rand -base64 32
```

Además, `JWT_EXPIRES_IN` se configuró para controlar la duración de la sesión. En desarrollo se puede usar un valor cómodo, pero en producción debe mantenerse una expiración razonable para reducir riesgos.

Paso 4. Variables de seguridad

JWT_SECRET se generó con alta entropía y se almacena únicamente en .env local.

```
terminal + editor · seguridad

$ openssl rand -base64 32
m1Jz...<censurado>...9sQ=

# SECURITY (JWT)
JWT_SECRET="*****"
JWT_EXPIRES_IN="15m"
JWT_REFRESH_EXPIRES_IN="7d"

# Importante: si JWT_SECRET falta, el backend avisa y usa fallback inseguro.
# En producción nunca debe usarse el fallback de desarrollo.
```

Figura 4: Configuración de variables de seguridad y token JWT.

3.5 Paso 5: configurar la app móvil Expo

La aplicación móvil necesita saber en qué URL se encuentra el backend. Para Expo, las variables que deben estar disponibles en el bundle del cliente utilizan el prefijo EXPO_PUBLIC_. Por eso se configuró:

Listing 4: Variable móvil principal.

```
EXPO_PUBLIC_API_URL="http://192.168.0.13:3002/api"
```

La URL cambia según el entorno de prueba:

- **Expo Web:** puede usar `http://127.0.0.1:3002/api`.
- **Emulador Android:** debe usar `http://10.0.2.2:3002/api`, porque `localhost` apunta al emulador.
- **Dispositivo físico:** debe usar la IP LAN del equipo, por ejemplo `http://192.168.0.13:3002/api`.

Paso 5. Configuración móvil Expo

La app móvil usa EXPO_PUBLIC_API_URL para comunicarse con apps/web.

```
editor · apps/mobile/.env

# API base para Expo / React Native
EXPO_PUBLIC_API_URL="http://192.168.0.13:3002/api"

# Alternativas según el dispositivo:
# Expo Web:      http://127.0.0.1:3002/api
# Android emulator: http://10.0.2.2:3002/api
# Físico en WiFi:  http://<IP-LAN>:3002/api

# Prefijo EXPO_PUBLIC_: requerido para exponer la variable al bundle móvil.
```

Figura 5: Configuración de EXPO_PUBLIC_API_URL para la aplicación móvil.

3.6 Paso 6: validar la configuración

Después de configurar las variables, se validó que Prisma pudiera leer el esquema y que la base estuviera sincronizada con sus migraciones. También se ejecutó TypeScript en la app móvil para confirmar que la configuración no rompiera el build.

Listing 5: Comandos de validación ejecutados.

```
npm exec -- prisma validate --schema apps/web/prisma/schema.prisma
npm exec -- prisma migrate status --schema apps/web/prisma/schema.prisma
npx tsc -p apps/mobile/tsconfig.json --noEmit
```

Paso 6. Validación técnica

Se verificó esquema Prisma, migraciones y compilación TypeScript.

```
terminal · validación

$ npm exec -- prisma validate --schema apps/web/prisma/schema.prisma
Prisma schema loaded from prisma/schema.prisma
The schema at prisma/schema.prisma is valid OK

$ npm exec -- prisma migrate status --schema apps/web/prisma/schema.prisma
Datasource "db": MySQL database "predia" at "127.0.0.1:3306"
Database schema is up to date!

$ npx tsc -p apps/mobile/tsconfig.json --noEmit
OK · sin errores de TypeScript
```

Figura 6: Validación técnica de Prisma, migraciones y TypeScript.

3.7 Paso 7: aplicar buenas prácticas

Finalmente se revisaron reglas básicas de seguridad para evitar exponer información sensible. El punto más importante es que `.env.example` sí puede compartirse porque contiene valores de ejemplo, pero los archivos con credenciales reales no deben subirse ni pegarse en reportes sin censura.

Paso 7. Buenas prácticas aplicadas

Se documentó qué se puede compartir y qué debe permanecer privado.

```
● ● ● checklist · seguridad

[OK] Documentar nombres de variables y propósito.
[OK] Enmascarar passwords, tokens y JWT_SECRET.
[OK] Usar .env.example como plantilla compartible.
[OK] Mantener .env.local fuera del control de versiones.
[OK] Separar URLs web, API y móvil.
[OK] Usar EXPO_PUBLIC_ solo para datos no secretos.
[OK] Validar Prisma y TypeScript después de configurar.

Resultado: PREDIA puede ejecutarse sin exponer credenciales.
```

Figura 7: Checklist de buenas prácticas aplicadas para variables de entorno.

4 Resultado final

La configuración final permite que PREDIA funcione como sistema integrado:

- El backend Next.js puede leer sus variables de base de datos, seguridad y URL pública.
- Prisma puede conectarse a MySQL mediante DATABASE_URL.
- La app móvil Expo puede consumir la API mediante EXPO_PUBLIC_API_URL.
- Los secretos reales permanecen fuera del código fuente y se documentan solamente con valores censurados.
- El proyecto conserva plantillas compartibles para que otros integrantes puedan configurar su propio entorno.

Resumen de validación

Las pruebas de configuración reportaron esquema Prisma válido, base de datos sincronizada con migraciones y compilación TypeScript móvil sin errores. Esto confirma que las variables de entorno configuradas son suficientes para ejecutar el flujo local de PREDIA.

5 Conclusión

La configuración de variables de entorno en PREDIA es una parte crítica del proyecto porque conecta tres elementos principales: backend web, base de datos y aplicación móvil.

Al separar las credenciales del código fuente, el sistema se vuelve más seguro, portable y fácil de ejecutar en distintos equipos.

El uso de `.env.example` facilita que nuevos integrantes conozcan qué variables deben configurar, mientras que `.env.local`, `apps/web/.env` y `apps/mobile/.env` permiten adaptar el proyecto al entorno real de desarrollo. La validación posterior con Prisma y TypeScript confirma que la configuración no solo está escrita, sino que funciona técnicamente.

Como práctica recomendada, ningún secreto debe mostrarse en reportes, capturas públicas o repositorios. Por esa razón, este informe documenta nombres, propósito y formato de las variables, pero censura valores sensibles como contraseñas, cadenas de conexión completas y claves JWT.